

COURSE MAJOR PROJECT:

ADVANCED ENTERPRISE NETWORK DESIGN & IMPLEMENTATION

Course: Computer Networking & Network Administration (Undergraduate Level)

Duration: 6 Weeks

Format: Take-home Group/Individual Project

Deliverables: Technical Explanation Report, Configuration/Code Files, Live Simulation Demo

INTRODUCTION TO THE PROJECT

1. CONTEXT & SCENARIO

TechViet Innovations (TVI), a leading multinational tech and logistics conglomerate based in Vietnam, is undergoing a massive digital transformation. Currently, TVI operates on a legacy, flat, traditional hub-and-spoke wide-area network utilizing outdated dynamic routing and perimeter-based security. With the rapid adoption of cloud services, hybrid work models, and AI-driven data analytics, their current infrastructure suffers from severe latency, frequent broadcast storms, and critical security vulnerabilities.

TVI is upgrading its entire infrastructure and has hired you as the Lead Network Architect. You must design and implement a next-generation enterprise network spanning three primary geographical regions:

- **Headquarters (HQ):** Ho Chi Minh City (Data Center, Executive Offices, 2,000+ users).
- **R&D & Engineering Campus:** Hanoi (High-bandwidth compute, AI labs, 1,500+ users).
- **Logistics & Distribution Hub:** Da Nang (IoT tracking devices, warehouse automation, 500+ users).

2. NECESSARY NEEDS FOR NETWORK DESIGN

The newly designed network must resolve existing bottlenecks and accommodate future growth. The core needs dictate:

- **High Availability & Redundancy:** Elimination of all single points of failure across Layer 2 and Layer 3.
- **Secure Remote Access:** Transitioning from legacy VPNs to Zero Trust Network Access (ZTNA) and Secure Access Service Edge (SASE) for a large hybrid workforce.
- **Optimized Traffic Flow:** Direct cloud access from branch offices without backhauling traffic through the HQ, reducing latency for SaaS applications.

- **Agility & Automation:** The ability to provision new VLANs, routing rules, and security policies programmatically via Infrastructure-as-Code (IaC) rather than manual CLI configuration.

3. GOALS AND SCALE

- **Scale:** Support an immediate base of 4,000+ internal users, 10,000+ IoT endpoints, and external guest networks, scalable to 10,000+ users by 2030.
- **Performance:** Guarantee 10/40 Gbps backbone throughput at HQ and 99.99% (Four Nines) uptime.
- **Convergence:** Seamlessly integrate voice, video, and data traffic over a unified infrastructure with strict Quality of Service (QoS) guarantees.

4. FEATURES OF THE ENTERPRISE NETWORK

- **Data Center Fabric:** Spine-and-Leaf architecture utilizing EVPN-VXLAN to provide highly scalable Layer 2 over Layer 3 overlay.
- **Wide Area Network:** SD-WAN deployment connecting HCMC, Hanoi, and Da Nang, utilizing dual-ISP broadband and MPLS underlays.
- **Campus LAN:** Collapsed core or three-tier architecture with dynamic VLAN assignment and robust loop prevention.
- **Security Posture:** Micro-segmentation, Next-Generation Firewalls (NGFW) at the edge, and strict Access Control Lists (ACLs) separating IoT, Guest, and Corporate traffic.

5. TARGET USERS

- **Executives & Finance:** Require maximum security, encrypted traffic, and absolute data privacy.
- **R&D Engineers (Hanoi):** Require massive east-west bandwidth for AI model training and access to secure private clouds.
- **Logistics & Warehouse Operations (Da Nang):** Rely on ultra-low latency Wi-Fi (Wi-Fi 6/7) for autonomous guided vehicles and real-time inventory scanners.
- **Remote Workforce:** Require seamless, authenticated access to internal applications via SASE/ZTNA frameworks.
- **Guests & Visitors:** Require internet-only access, strictly isolated from internal corporate resources.

6. INTEGRATED TECHNIQUES AND TECHNOLOGIES

Students must research and integrate the following trending enterprise technologies:

- **Routing & Switching:** OSPFv3, MP-BGP, EVPN-VXLAN, IPv4/IPv6 Dual-Stack.
- **WAN Technologies:** SD-WAN concepts, IPsec VPN tunnels, NAT/PAT.
- **Security:** Port Security, DHCP Snooping, Dynamic ARP Inspection (DAI), Zone-based Firewalls, ZTNA principles.
- **Automation & Programmability:** Python (Netmiko/NAPALM) or Ansible to push baseline configurations and collect telemetry.

- **Wireless:** WLC (Wireless LAN Controller) integration with CAPWAP.

7. DESIGN & TECHNICAL REQUIREMENTS

- **Redundancy:** FHRP (HSRP/VRRP) must be configured for default gateway redundancy. Core links must utilize EtherChannel (LACP).
- **IP Management:** Hierarchical IPv4 subnetting (VLSM) and IPv6 deployment. DHCP servers must be centralized at HQ with DHCP Relay at branches.
- **Routing:** OSPF for internal site routing (Underlay); BGP for inter-site and external ISP routing (Overlay).
- **QoS:** Voice traffic (VoIP) must be prioritized over all WAN links.

8. IMPLEMENTATION LAB REQUIREMENTS

The project must be virtually simulated and proven functional.

- **Platforms Allowed:** EVE-NG, GNS3, or Cisco Modeling Labs (CML). Cisco Packet Tracer may only be used if EVE-NG/GNS3 hardware requirements cannot be met, but requires instructor pre-approval due to limited protocol support (e.g., EVPN/BGP limitations).
- **Images:** Cisco vIOS, Arista vEOS, or Juniper vSRX/vQFX.
- **Compute:** A minimum of 16GB RAM is recommended for running the full topology in EVE-NG.

9. 10-STEP IMPLEMENTATION PROCESS & EVALUATION CRITERIA (10 POINTS TOTAL)

Step	Design Phase	Expected Outcome	Points
1	Requirement Analysis & Logical Topology	A comprehensive Visio/Draw.io diagram mapping out HQ, branches, and ISP clouds with clear demarcation points.	1.0
2	IP Addressing & Subnetting Strategy	An Excel/Markdown IP schema detailing VLSM for IPv4 and hierarchical IPv6 allocations across all sites.	1.0
3	Layer 2 Infrastructure & STP	Configured VLANs, Trunking (802.1Q), EtherChannels, and Spanning Tree (RSTP/MSTP) with Root Bridge enforcement.	1.0
4	First Hop Redundancy & IP Services	Functional HSRP/VRRP for gateway redundancy, and centralized DHCP/DNS functioning across routed boundaries.	1.0

Step	Design Phase	Expected Outcome	Points
5	Underlay Routing (IGP)	Fully converged OSPF/EIGRP within each site. All loopbacks must be reachable within the local site.	1.0
6	Overlay Routing (BGP) & WAN Edge	eBGP established with ISPs; iBGP/EVPN or IPsec tunnels configured connecting HQ to Hanoi and Da Nang.	1.0
7	Security & Hardening	Implemented ACLs isolating Guest from Corp, NAT overload to the internet, and basic Layer 2 security (DHCP Snooping/DAI).	1.0
8	Quality of Service (QoS)	Verified QoS policies marking and prioritizing simulated VoIP traffic across the WAN interfaces.	1.0
9	Network Automation	A functional Python script or Ansible playbook that automatically configures VLANs or backups device configurations.	1.0
10	Testing, Verification & Documentation	Successful end-to-end ping/trace tests, professional Technical Explanation Report, and clear operational documentation.	1.0

10. PROJECT TIMELINE & SUBMISSION GUIDE

You have exactly 6 weeks to complete this major project. Adhere strictly to the following submission schedule.

Week	Milestone	Deliverables
Week 1	Design & IP Schema	Submit Logical Network Diagram and IP Addressing Plan for approval.
Week 2	Layer 2 & IP Services	Build lab topology. Configure VLANs, LACP, STP, HSRP, and DHCP.

Week	Milestone	Deliverables
Week 3	Internal Routing (Underlay)	Implement OSPF across all sites. Ensure internal reachability.
Week 4	WAN, BGP & Security	Connect branches via BGP/IPsec. Implement NAT and edge ACLs.
Week 5	Automation & QoS	Apply QoS policies. Write and test automation scripts.
Week 6	Final Submission & Demo	Submit final Technical Report, configuration files, and present Live Demo.

FINAL SUBMISSION REQUIREMENTS:

1. **Technical Explanation Report (PDF):** Must include executive summary, design methodologies, IP schemas, topology diagrams, security policies, and testing results.
2. **Configuration / Code Files:** Exported startup-configs of all routers/switches, and all Ansible .yaml or Python .py scripts.
3. **Simulation Demo:** A 15-minute live demonstration proving end-to-end connectivity, failover simulation (shutting down a core link), and executing the automation script.

Required Submission Content:

Students must submit a .zip file or a link to a private GitHub repository containing the following:

1. **Source Code:**
 - All client and server scripts and a README.md file with strict, step-by-step instructions on how to install dependencies, run the server, and connect the clients.
2. **Technical Explanation Report (PDF):**
 - **System Architecture:** A high-level diagram showing how clients and the server interact.
 - **Protocol Specification:** A breakdown of the custom packet/message structures designed for the application.
 - **Sequence Diagram:** A flow chart showing the lifecycle of a message and a file transfer.
 - **Challenges & Solutions:** A brief 1-page reflection on network challenges faced (e.g., resolving deadlocks, handling packet fragmentation) and how they were solved.

SUGGESTED DEADLINE: **SUNDAY, NOVEMBER 17, 2026, AT 11:59 PM.**

TECHNICAL EXPLANATION REPORT: EVALUATION RUBRIC

This rubric evaluates the final written documentation of the TechViet Innovations (TVI) network design. The report accounts for a significant portion of the final project grade and must reflect executive-level professional engineering standards.

Evaluation Criteria	Weight	Excellent (90–100%)	Proficient (75–89%)	Developing (60–74%)	Unacceptable (<60%)
1. Executive Summary & Business Alignment	10%	Flawlessly translates TVI's business needs (hybrid work, AI/IoT scaling, latency) into technical goals. The summary is concise, persuasive, and executive-ready.	Adequately addresses TVI's needs but may lean too heavily on technical jargon in the executive section. Lacks minor business context.	Summarizes the project loosely without directly addressing TVI's specific branches or business bottlenecks.	Missing summary or fails entirely to connect the technical design to the assigned business scenario.
2. Architecture & Topology Diagrams	15%	Visio/Draw.io diagrams are immaculate. Logical/physical layers, demarcation points, VLANs, and IP boundaries are clearly and professionally labeled.	Diagrams are complete and mostly accurate. Some minor labels (e.g., interface IDs, specific subnets) are missing or cluttered.	Diagrams are present but difficult to read, lack standard iconography, or fail to clearly distinguish between L2 and L3 domains.	Diagrams are hand-drawn, entirely missing, or contain fatal architectural misrepresentations.
3. IP Addressing Schema (IPv4/IPv6)	15%	Perfect application of VLSM and IPv6 hierarchical subnetting. Spreadsheets/tables clearly show scalable allocation for 10k+ endpoints	IP schema is functional and accurate but may lack optimal summarization at the branch edges or has minor inefficiencies.	Schema contains overlapping subnets, incorrect subnet masks, or fails to implement IPv6	Fatal subnetting errors causing routing failures. IP allocation is completely arbitrary.

Evaluation Criteria	Weight	Excellent (90–100%)	Proficient (75–89%)	Developing (60–74%)	Unacceptable (<60%)
		without massive IP waste.		alongside IPv4.	
4. Core Infrastructure & Technology Rationale	20%	Exceptional justification for all chosen protocols (OSPF, BGP, EVPN-VXLAN, STP, HSRP). Deep dive into <i>why</i> these technologies solve TVI's exact scale requirements.	Solid explanation of configurations and protocols, but relies on generic descriptions rather than tying them strictly to TVI's specific architecture.	Lists configurations without explaining the rationale. Fails to adequately explain the interaction between underlay and overlay networks.	Missing justification. Configurations in the report do not match the assigned requirements (e.g., used RIP instead of OSPF/BGP).
5. Security, QoS & Automation Strategy	15%	Thoroughly documents micro-segmentation, ACLs, ZTNA approach, QoS markings for VoIP, and includes a detailed explanation of the automation scripts (Python/Ansible).	Discusses security, QoS, and automation sufficiently, but lacks depth on specific ACL logic or script execution workflows.	Vague mention of security policies. QoS is poorly explained. Automation script is either missing from the report or completely unexplained.	Ignores network security, QoS, and programmable automation requirements entirely.
6. Testing, Verification & Failover	15%	Provides robust evidence of validation (e.g., CLI outputs of show ip route,	Includes basic ping and traceroute evidence, but lacks comprehensive	Minimal testing evidence. Relies solely on stating "it works"	No verification data included. The report provides no proof that the lab simulation

Evaluation Criteria	Weight	Excellent (90–100%)	Proficient (75–89%)	Developing (60–74%)	Unacceptable (<60%)
		tracertoute, ping). Failover scenarios (e.g., shutting down HQ core links) are documented and proven successful.	proof of high availability/failover convergence.	without providing CLI screenshots or command outputs.	was successfully executed.
7. Formatting, Clarity & Professionalism	10%	English is technically precise, grammatically flawless, and highly readable. Excellent use of headers, TOC, bullet points, and citations.	Good formatting and generally clear language. A few minor grammatical errors or formatting inconsistencies that do not impede understanding.	Noticeable grammatical issues. Formatting is blocky, lacks a Table of Contents, or is difficult to navigate.	Highly unprofessional, heavily plagiarized/AI-generated without editing, or unreadable due to severe language/formatting issues.

GRADING NOTES FOR THE INSTRUCTOR:

- **Plagiarism & Code Integrity:** The Python/Ansible scripts and configuration files cross-referenced in this report must be run through a plagiarism checker. Identical device naming conventions or script hashes across different student groups trigger an automatic review.
- **Simulation Mismatch:** If the report claims a successful failover (Criterion 6), but the live simulation demo fails to replicate it, dock the report's verification score by up to 10 points.

STEP 9: NETWORK AUTOMATION REQUIREMENTS & STARTER CODE

For Step 9 of the TechViet Innovations (TVI) project, students must move beyond manual CLI entry and implement Infrastructure-as-Code (IaC) or programmatic device management. The goal is to enforce operational consistency, automate repetitive provisioning, and implement automated backup/compliance workflows across the HCMC HQ, Hanoi, and Da Nang network nodes.

1. DETAILED AUTOMATION REQUIREMENTS

Students can choose either **Python (using Netmiko/NAPALM)** or **Ansible**. The submission must fulfill three core automation tasks:

1. Inventory & Credentials Management:

- Infrastructure hosts, management IP addresses, device types (cisco_ios, arista_eos, etc.), and SSH ports must be defined externally in a structured inventory file (inventory.yaml or Ansible hosts file).
- Plaintext credentials inside scripts are strictly forbidden. Students must use environment variables, interactive getpass, or Ansible Vault.

2. Automated Baseline & VLAN Provisioning:

- Programmatically push baseline configurations (AAA, NTP, Syslog, banner) across core and access switches.
- Dynamically create and name standard corporate VLANs (VLAN 10: Executive, VLAN 20: Engineering, VLAN 30: Logistics, VLAN 99: Management) based on the input payload.

3. State Capture & Configuration Backup:

- Automatically execute show running-config and show ip interface brief across all nodes before and after applying changes.
- Save timestamped output files (<hostname>_<timestamp>_backup.cfg) into a designated /backups/ directory.
- Log all successful executions and SSH error exceptions (e.g., authentication failure, connection timeout) to an automation.log file.

2. STARTER CODE IMPLEMENTATION

Option A: Python Implementation (netmiko + pyyaml)

Students choosing Python must structure their project as follows:

Plaintext

automation_project/

| — inventory.yaml

| — auto_provision.py

| — backups/

| — logs/

inventory.yaml

YAML

devices:

```
- hostname: HCMC-CORE-SW01
  ip: 192.168.1.10
  device_type: cisco_ios
- hostname: HANOI-RND-SW01
  ip: 192.168.2.10
  device_type: cisco_ios
- hostname: DANANG-LOG-SW01
  ip: 192.168.3.10
  device_type: cisco_ios
```

auto_provision.py

Python

```
import os
import yaml
import logging
from datetime import datetime
from getpass import getpass
from netmiko import ConnectHandler, NetmikoTimeoutException, NetmikoAuthenticationException
```

1. Setup Logging

```
logging.basicConfig(
    filename='logs/automation.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
```

2. Configuration Commands Payload

```
CONFIG_COMMANDS = [
    'ntp server 10.10.10.50',
    'banner motd ^C Unauthorized access to TVI Corporate Infrastructure is prohibited. ^C',
    'vlan 10', 'name EXEC_FINANCE',
```

```
'vlan 20', 'name RND_ENGINEERING',  
'vlan 30', 'name LOGISTICS_OPS',  
'vlan 99', 'name MANAGEMENT'  
]
```

```
def load_inventory(filepath='inventory.yaml'):  
    with open(filepath, 'r') as f:  
        return yaml.safe_load(f)['devices']
```

```
def run_automation():  
    # Prompt for credentials once  
    username = input("Enter SSH Username: ")  
    password = getpass("Enter SSH Password: ")
```

```
os.makedirs('backups', exist_ok=True)  
os.makedirs('logs', exist_ok=True)  
devices = load_inventory()
```

```
for dev in devices:  
    device_params = {  
        'device_type': dev['device_type'],  
        'host': dev['ip'],  
        'username': username,  
        'password': password,  
        'timeout': 10,  
    }  
  
    hostname = dev['hostname']  
    print(f"\n[*] Connecting to {hostname} ({dev['ip']})...")  
  
    try:  
        net_connect = ConnectHandler(**device_params)
```

```
net_connect.enable()
```

```
# --- TASK 1: Backup Configuration ---
```

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
backup_filename = f"backups/{hostname}_{timestamp}.cfg"
```

```
running_config = net_connect.send_command('show running-config')
```

```
with open(backup_filename, 'w') as backup_file:
```

```
    backup_file.write(running_config)
```

```
print(f" [+] Backup saved to {backup_filename}")
```

```
# --- TASK 2: Push Configurations ---
```

```
print(f" [+] Deploying baseline configuration and VLANs...")
```

```
output = net_connect.send_config_set(CONFIG_COMMANDS)
```

```
logging.info(f"Successfully configured {hostname}: \n{output}")
```

```
# --- TASK 3: Save Running Config to Startup ---
```

```
net_connect.send_command('write memory')
```

```
print(f" [+] Saved startup configuration.")
```

```
net_connect.disconnect()
```

```
except NetmikoTimeoutException:
```

```
    msg = f"[-] Connection timed out for {hostname} ({dev['ip']})"
```

```
    print(msg)
```

```
    logging.error(msg)
```

```
except NetmikoAuthenticationException:
```

```
    msg = f"[-] Authentication failed for {hostname} ({dev['ip']})"
```

```
    print(msg)
```

```
    logging.error(msg)
```

```
except Exception as e:
```

```
    msg = f"[-] Unexpected error on {hostname}: {str(e)}"
```

```
print(msg)
logging.error(msg)
```

```
if __name__ == "__main__":
    run_automation()
```

Option B: Ansible Implementation

Students using Ansible should use standard network modules (cisco.ios.ios_config, cisco.ios.ios_facts).

hosts Inventory File

Ini, TOML

[hcmc]

HCMC-CORE-SW01 ansible_host=192.168.1.10

[hanoi]

HANOI-RND-SW01 ansible_host=192.168.2.10

[danang]

DANANG-LOG-SW01 ansible_host=192.168.3.10

[tvi_switches:children]

hcmc

hanoi

danang

[tvi_switches:vars]

ansible_network_os=cisco.ios.ios

ansible_user=admin

ansible_connection=network_cli

deploy_baseline.yml Playbook

YAML

- name: TVI Network Automation & Provisioning Playbook

hosts: tvi_switches

gather_facts: no

vars:

vlan_list:

- { id: 10, name: 'EXEC_FINANCE' }
- { id: 20, name: 'RND_ENGINEERING' }
- { id: 30, name: 'LOGISTICS_OPS' }
- { id: 99, name: 'MANAGEMENT' }

tasks:

- name: Ensure output directory exists

ansible.builtin.file:

path: "./backups"

state: directory

delegate_to: localhost

- name: Capture Running Configuration

cisco.ios.ios_command:

commands:

- show running-config

register: running_config

- name: Save Running Configuration locally

ansible.builtin.copy:

content: "{{ running_config.stdout[0] }}"

dest: "./backups/{{ inventory_hostname }}_{{ lookup('pipe', 'date +%Y%m%d_%H%M%S') }}.cfg"

delegate_to: localhost

- name: Provision Corporate VLANs

cisco.ios.ios_vlans:

config:

- vlan_id: "{{ item.id }}"

name: "{{ item.name }}"

state: active

loop: "{{ vlan_list }}"

- name: Configure NTP and Banner

cisco.ios.ios_config:

lines:

- ntp server 10.10.10.50

- banner motd ^C Unauthorized access to TVI Corporate Infrastructure is prohibited. ^C

3. Student Evaluation Criteria for Step 9

1. **Execution Success (50%):** Script/playbook executes cleanly against EVE-NG/GNS3 lab topologies without manual intervention.
2. **Error Handling & Idempotency (30%):** Script handles dead nodes smoothly without crashing and skips unneeded re-configurations.
3. **Code Quality & Security (20%):** Modular code structure, clear comments, proper logging, and no hardcoded plaintext passwords.

VIRTUAL LAB SETUP GUIDE: EVE-NG / GNS3

To execute the 6-week **TechViet Innovations (TVI)** network engineering project, students must deploy a high-fidelity virtual emulation environment. This guide outlines the hardware requirements, hypervisor setup, image installation, and automation container configuration required to run the full HQ–Hanoi–Da Nang topology.

PHASE 1: HARDWARE & HYPERVISOR PREREQUISITES

Running multi-site enterprise topologies (including vIOS, vIOS-L2, BGP/EVPN nodes, and an automation control node) requires dedicated hypervisor hardware acceleration.

Minimum vs. Recommended System Hardware

Component	Minimum Specification	Recommended Specification
CPU	8 Cores (Intel VT-x / AMD-V enabled in BIOS)	12+ Cores (AMD Ryzen / Intel Core i7/i9 or Xeon)
RAM	16 GB DDR4	32 GB – 64 GB DDR4/DDR5
Storage	100 GB Free SSD / NVMe	250 GB+ NVMe SSD
Hypervisor	VMware Workstation Pro / Player 17+, VMware ESXi, or Proxmox VE	VMware Workstation Pro / Proxmox VE

⚠ **Crucial Setting:** Nested Virtualization must be explicitly enabled in your hypervisor settings for the virtual appliance (e.g., “*Virtualize Intel VT-x/EPT or AMD-V/RVI*” in VMware).

PHASE 2: HYPERVISOR APPLIANCE DEPLOYMENT

OPTION A: EVE-NG (RECOMMENDED FOR MULTI-SITE TOPOLOGY)

- Download:** Obtain the latest EVE-NG Community ISO or EVE-NG Professional / Freemium OVA file from the official download portal.
- Import OVA:** In VMware Workstation / Proxmox, select **Open / Import** and load the EVE-NG .ova file.
- VM Resource Allocation:**
 - Processor Cores: Minimum 8 cores. Check **Virtualize Intel VT-x/EPT**.
 - Memory: Minimum 16,384 MB (16 GB).
 - Network Adapter 1: Set to **Bridged** (or **NAT** if on a restricted DHCP Wi-Fi connection).
- First Boot Configuration:**

- Boot the VM, log in with default credentials (root / eve).
- Follow the wizard: Set static IP or DHCP, define DNS (8.8.8.8), and leave default hostname settings.
- Access the web dashboard via `http://<EVE_IP_ADDRESS>` using credentials admin / eve.

OPTION B: GNS3 SETUP

1. **Download:** Install the GNS3 All-in-One GUI desktop client and the matching **GNS3 VM** image for VMware Workstation.
2. **Import GNS3 VM:** Import the .ova into VMware and assign 16 GB RAM and 8 vCPUs.
3. **Bind GUI to VM:** Open GNS3 GUI → **Edit** → **Preferences** → **GNS3 VM** → Select *Enable the GNS3 VM* and choose VMware Workstation.

PHASE 3: SOURCING & UPLOADING CISCO / NETWORK IMAGES

Students must upload image files via SFTP (using FileZilla or WinSCP) to the virtual server.

Required Image Profiles

1. **Cisco vIOS-L2 (Switching):** `viosl2-adventerprisek9-m.vmdk.SSA.152-4.0.55a`
2. **Cisco vIOS-L3 (Routing):** `vios-adventerprisek9-m.spa.152-4.M3`
3. **Linux / Network Automation Node:** Alpine Linux Docker image or Ubuntu Server minimal.

EVE-NG Image Directory Structure & Commands

SFTP into EVE-NG at `/opt/unetlab/addons/qemu/`. Create folders matching exact vendor naming conventions:

Bash

1. SSH into EVE-NG server as root

`cd /opt/unetlab/addons/qemu/`

2. Create directory for vIOS Router

`mkdir vios-152M3`

Upload 'vios-adventerprisek9-m.spa.152-4.M3.vmdk' into this folder and rename it `hda.qcow2`:

`mv vios-adventerprisek9-m.spa.152-4.M3.vmdk virtioa.qcow2`

3. Create directory for vIOS Switch

`mkdir viosl2-1524`

Upload 'viosl2-adventerprisek9-m.SSA.152-4.0.55a.vmdk' into this folder and rename it:

`mv viosl2-adventerprisek9-m.SSA.152-4.0.55a.vmdk virtioa.qcow2`

4. FIX PERMISSIONS (Mandatory step in EVE-NG after uploading any image)

```
/opt/unetlab/wrapper/unl_wrapper -a fixpermissions
```

Phase 4: Network Automation Control Node Setup

To fulfill Step 9 (Ansible/Python execution), you must add an automation host inside the lab that can reach all router/switch management IP addresses.

Option A: Docker Network Automation Container (GNS3 / EVE-NG)

In EVE-NG, pull the built-in Network Automation container:

Bash

```
# SSH into EVE-NG CLI and pull Docker image
```

```
docker pull networktocode/network-automation
```

In the EVE-NG Web Interface: Right-click lab canvas → **Node** → Select **Docker** → Image: networktocode/network-automation.

Option B: Standalone Ubuntu Control Node

1. Add an **Ubuntu / Linux Workstation** node connected to a **Management Cloud (Management / NAT interface)**.
2. Open terminal on the Linux node and install the required Python dependencies:

Bash

```
sudo apt update && sudo apt install -y python3 python3-pip git ansible
```

```
pip3 install netmiko napalm pyyaml paramiko
```

Phase 5: Building & Verifying the TVI Project Topology

1. **Topology Wireup:**
 - **HCMC HQ Node:** Place 1x Core Switch (vIOS-L2), 2x Distribution Switches, 1x Edge Router (vIOS).
 - **Hanoi Branch Node:** Place 1x Router (vIOS) + 2x Switches (vIOS-L2).
 - **Da Nang Branch Node:** Place 1x Router (vIOS) + 1x Switch (vIOS-L2).
 - **ISP Cloud:** Connect HQ, Hanoi, and Da Nang Routers to an ISP node running BGP.
 - **Management/Automation Node:** Connect the eth0 interface of the Network Automation host to a dedicated Management VLAN (VLAN 99) or Management Cloud on all devices.
2. **Basic Baseline Connectivity Check:**
 - Configure basic IP addressing and SSH on all switches/routers:

Cisco CLI

```
crypto key generate rsa modulus 2048
```

```
username admin privilege 15 secret Cisco123!
```

```
line vty 0 4
```

```
transport input ssh
```

```
login local
```

- From the Network Automation host, test SSH reachability:

Bash

```
ssh admin@192.168.1.10
```

[Network Automation Lab Setup Guide](#)

This video provides a complete walk-through for building a network automation lab environment using VMware and GNS3 VM, making it an ideal visual reference for completing this step.

PRACTICAL TROUBLESHOOTING CHECKLIST FOR COMMON LAB ERRORS

1. PERMISSION & IMAGE DIRECTORY ERRORS (EVE-NG / GNS3)

Symptom	Primary Root Cause	Corrective Action / CLI Fix
Node turns blue then gray (fails to launch instantly).	Incorrect file permissions after SFTP transfer.	Run the native EVE-NG permission fix tool: <code>/opt/unetlab/wrapper/unl_wrapper -a fixpermissions</code>
Node throws "Image not found" error on start.	Incorrect directory or image filename syntax.	Ensure folder name in <code>/opt/unetlab/addons/qemu/</code> matches expected prefixes (e.g., <code>vios-152M3</code> or <code>viosl2-1524</code>). The disk image file inside must be renamed to <code>virtioa.qcow2</code> .
GNS3 VM shows <code>qemu-system-x86_64: permission denied</code> .	Missing KVM execution rights for the gns3 user.	Run: <code>sudo usermod -aG kvm \$USER</code> <code>sudo chmod 666 /dev/kvm</code>

2. NODE BOOT LOOPS & HUNG BOOT CYCLES

- **Symptom:** Cisco vIOS / vIOS-L2 or vSRX gets stuck in an infinite boot cycle or freezes at Booting 'Virtio-Block drive...'.
- **Fix Checklist:**
 1. **Verify Nested Virtualization:** Open your host hypervisor (VMware / Proxmox). Ensure **Virtualize Intel VT-x/EPT or AMD-V/RVI** is checked in CPU settings for the VM.
 2. **Wipe Node NVRAM:** In EVE-NG/GNS3 GUI, right-click the stuck node → **Stop**, then right-click → **Wipe** (clears corrupt temporary nvram/disk overlay state).
 3. **Check Resource Allocation:** Allocate at least 1024 MB RAM for vIOS-L2 and 512 MB for vIOS-L3 nodes. Lower allocations cause silently aborted Linux kernel boots inside the image.

3. SSH TIMEOUTS & CONNECTION FAILURES FROM AUTOMATION HOST

Plaintext

[Error] NetmikoTimeoutException: Connection to device timed out on port 22

- **Fix Checklist:**

1. **Verify Line VTY Settings on Router/Switch:**

Cisco CLI

configure terminal

ip domain name tvi.lab

crypto key generate rsa modulus 2048

username admin privilege 15 secret Cisco123!

line vty 0 4

transport input ssh

login local

2. **Clear SSH Known Hosts Conflicts:** If IP addresses were reused from previous lab instances, clear stale keys on your Python/Ansible control node:

Bash

ssh-keygen -f "~/.ssh/known_hosts" -R "192.168.1.10"

3. **Verify L2/L3 Management Network:** Confirm the automation container/VM interface is mapped to the correct EVE-NG Cloud0 (Management / Bridged) interface and can ping the target IP.

4. HIGH CPU USAGE & OUT-OF-MEMORY (OOM) CRASHES

- **Symptom:** EVE-NG/GNS3 interface stops responding, or Linux kernel kills QEMU instances (Out of memory: Kill process).

- **Fix Checklist:**

1. **Stagger Node Booting:** Never click "Start all nodes" simultaneously on topologies exceeding 10 devices. Highlight and start nodes in batches of 3–5 with 30-second delays.
2. **Check Memory Footprint via CLI:**

Bash

free -h

top -o %MEM

3. **Disable Unused Interfaces:** On high-density topologies, issue shutdown on unused switchports to reduce internal CPU overhead within vIOS-L2 instances.

5. LINK FLAPPING & STP SPANNING-TREE TOPOLOGY CHANGE STORMS

- **Symptom:** High latency between lab nodes; terminal console flooded with %SW_MATC-4-MACFLAP or %PORT_SECURITY-2-PSECURE_VIOLATION.

- **Fix Checklist:**

1. **Enforce STP Root Bridge Priorities:** Prevent default priority ties across switches. Set HQ Core Switch explicitly as primary root:

Cisco CLI

```
spanning-tree mode rapid-pvst
```

```
spanning-tree vlan 1-4094 root primary
```

2. **Disable Keepalives on Point-to-Point Routed Links:** On vIOS interface links connecting L3 routers, turn off keepalives to prevent false link-down alerts caused by QEMU emulation delays:

Cisco CLI

```
interface GigabitEthernet0/1
```

```
no keepalive
```

PRE-SUBMISSION VERIFICATION CHECKLIST

1. LAB TOPOLOGY & LAYER 2 INFRASTRUCTURE

- [] **Device Naming & Hostnames:** Every switch and router adheres strictly to the project naming convention (e.g., HCMC-CORE-SW01, HANOI-RND-RTR01, DANANG-LOG-SW01).
- [] **Spanning Tree Protocol (STP):** Rapid PVST+ or MSTP is enabled across all switching domains. Primary and secondary root bridges are explicitly forced via priority configurations (spanning-tree vlan X root primary).
- [] **VLAN & Trunking Standard:** 802.1Q trunking is configured on inter-switch links with unused ports assigned to an isolated Blackhole VLAN (VLAN 999) and administrative shutdown applied.
- [] **Link Aggregation:** Multi-chassis or EtherChannel links use LACP (channel-group X mode active) rather than static PAgP or manual modes.

2. IP SCHEMA & ROUTING PROTOCOLS

- [] **Addressing & Subnetting:** IPv4 VLSM allocations match the submitted IP plan without subnet overlaps. IPv6 global unicast addresses (GUA) are assigned alongside IPv4 on dual-stacked interfaces.
- [] **Underlay Routing (IGP):** OSPFv3 or multi-area OSPF is fully converged within each site. All loopback interfaces (Loopback0) are reachable site-wide, and passive interfaces are configured on non-routing user subnets.
- [] **Overlay Routing (BGP/WAN):** eBGP/iBGP peering is established across the ISP boundary. Route summarization is enabled at branch edges to minimize global routing table sizes.
- [] **EVPN-VXLAN / IPsec Verification:** Overlay tunnels between HQ (HCMC) and branches (Hanoi/Da Nang) show UP/UP state with active data plane encapsulation.

3. SERVICES, SECURITY & QOS

- [] **Gateway Redundancy:** HSRP or VRRP is verified on access layer virtual gateways with preemption enabled on primary nodes (priority 110).
- [] **DHCP & DNS Services:** Dynamic IP allocation works across routed boundaries via DHCP Relay (ip helper-address) targeting centralized HQ servers.
- [] **Network Address Translation (NAT):** PAT/NAT Overload is active on edge routers, correctly translating internal private scopes to public WAN IPs for internet access.
- [] **Access Control & Hardening:** Zone-based or access list policies block unauthorized access between Corporate, Guest, and IoT subnets. Insecure protocols (Telnet, HTTP, SNMPv1/v2c) are disabled in favor of SSHv2, HTTPS, and SNMPv3.
- [] **Quality of Service (QoS):** CoS/DSCP markings (EF for Voice, AF31 for Video) are set, and queuing policies prioritize VoIP traffic across congested WAN links.

4. NETWORK AUTOMATION CODE (STEP 9)

- [] **Zero Hardcoded Secrets:** Credentials are introduced via environment variables, getpass(), or encrypted Ansible Vault files—no plaintext passwords exist in .py or .yaml files.

- [] **Externalized Inventory:** Device lists, IP addresses, and SSH ports are defined in inventory.yaml or an Ansible hosts file.
- [] **Idempotency & Error Exception Handling:** Python scripts incorporate try/except blocks (handling NetmikoTimeoutException / AuthenticationException), and Ansible playbooks execute without unnecessary state changes on repeated runs.
- [] **Configuration Backups & Logging:** Automated output captures running configurations into /backups/ with ISO timestamps, and execution events write directly to /logs/automation.log.

5. TECHNICAL EXPLANATION REPORT

- [] **Executive Summary:** Connects TVI's business expansion requirements directly to chosen network architectural decisions.
- [] **Diagram Clarity:** High-resolution logical and physical topologies (Visio/Draw.io) explicitly label interfaces, IP subnets, VLAN IDs, and trust zones.
- [] **Verification Evidence:** CLI output screenshots (show ip route, show ip bgp summary, ping/traceroute verification, and failover tests) validate operational stability.
- [] **Format Compliance:** Document is formatted as a single PDF containing a Table of Contents, page numbers, technical justifications, and zero unedited AI-generated prose.

6. SUBMISSION PACKAGE & LIVE DEMO PREPARATION

- [] **Archive Structure:** Submission ZIP archive contains three organized sub-directories:

Plaintext

TVI_Project_GroupXX/

└── Report/

| └── TVI_Technical_Explanation_Report.pdf

└── Configurations/

| └── HQ_Startup_Configs/

| └── Hanoi_Startup_Configs/

| └── DaNang_Startup_Configs/

└── Automation/

└── inventory.yaml

└── auto_provision.py (or deploy_baseline.yml)

└── backups/

- [] **Demo Failover Simulation:** Team can demonstrate live convergence in under 3 minutes when a primary core link or gateway is manually shut down (shutdown command).